

Estimator Platform

Design system and user experience documentation
across four estimation contexts

4

CONTEXTS

17

COMPONENTS

33

DESIGN TOKENS

169

PAGES

Platform Overview

A unified estimation platform serving four distinct business domains

The Service Estimator is a multi-context platform enabling teams to create, manage, and approve cost estimates for services and products. All four domains share a single application with common design infrastructure — one codebase, one design system, four experiences.

FOUR CONTEXTS

SERVICE ESTIMATOR

Custom Module Estimation

Complex project scoping with stages, milestones, resource planning, and multi-step approval workflows. The most feature-rich context.

PRODUCT ESTIMATOR

Catalog-Based Selection

Product catalog browsing with addon configuration, environment setup, and streamlined reporting output.

SERVICE MANAGEMENT

Reusable Packages

Pre-built service packages with pricing, discount management, and one-click package cloning for speed.

SMS GATEWAY

Tiered Usage Pricing

SMS and voice usage pricing with tiered rates, prorated billing, and custom price overrides per client.

50

ROUTES

169

PAGES

17

COMPONENTS

33

TOKENS

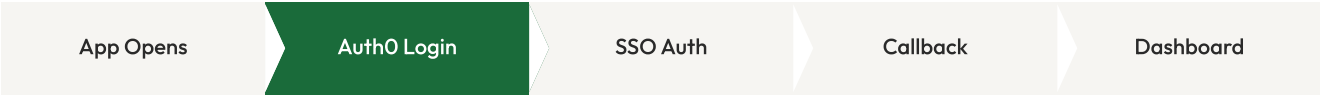
KEY DESIGN DECISION

All four contexts share the same design tokens, component library, and interaction patterns. Users who learn one context navigate all others without relearning any UI patterns — dramatically lowering adoption friction across teams.

Authentication & Onboarding

Secure, seamless login with enterprise SSO integration

LOGIN FLOW



STEP-BY-STEP EXPERIENCE

STEP	USER SEES	DESIGN RATIONALE
Initial Visit	Loading animation (6 dots)	Brief indicator while auth state is resolved
Login Page	Auth0 hosted login UI	Enterprise SSO — no custom form to maintain or secure
Callback	Spinner + "Processing login..."	Reassures the user during the auth redirect window
Error State	Error message + "Go Home"	Clear recovery path — user is never left stranded
Success	Dashboard context selector	Clean, focused first interaction — no unnecessary steps

DESIGN DECISIONS

- ✓ Single sign-on via Auth0 — no separate credentials to manage per user
- ✓ Token persistence in localStorage — stay logged in across tabs and page refreshes
- ✓ Deep-link preservation — users return to their intended page after login completes
- ✓ Permission-based rendering — admin features hidden at the component level for regular users
- ✓ Automatic token injection — every API call is authenticated transparently via interceptor

SECURITY + UX BALANCE

The platform supports dual identity providers — Auth0 primary, Okta as fallback — for enterprise flexibility. The user experience is identical regardless of which provider handles authentication in the background.

Service Estimation Workflow

Progressive 6-step wizard for complex estimation tasks

WIZARD NAVIGATION

STEP 01 Module Details	STEP 02 Cart	STEP 03 Stages	STEP 04 Milestones	STEP 05 Resources	STEP 06 Summary
---------------------------	-----------------	-------------------	-----------------------	----------------------	--------------------

STEP DETAILS

STEP	USER ACTION	UX PATTERN
1. Module Details	Define modules and sub-modules	Form + expandable card list
2. Cart	Review selections, configure tech stack	Shopping cart pattern
3. Stages	Break modules into delivery stages	Ordering interface
4. Milestones	Define payment milestones	Kanban drag-and-drop board
5. Resources	Allocate team members and hours	Editable data table
6. Summary	Review charts, export report	Read-only report with export action

Smart Branching

T&M contracts skip the Milestones step entirely — only Fixed Fee estimates require payment milestone definition.

Progress Persistence

Users can leave mid-workflow and return to their last completed step. No data is lost between sessions.

BUSINESS VALUE

- ✓ Reduces estimation time through structured, step-by-step guidance
- ✓ Prevents configuration errors via per-step validation before progression
- ✓ Supports both Fixed Fee and T&M contract types within a single unified workflow
- ✓ Approval trail provides accountability and full revision history for stakeholders

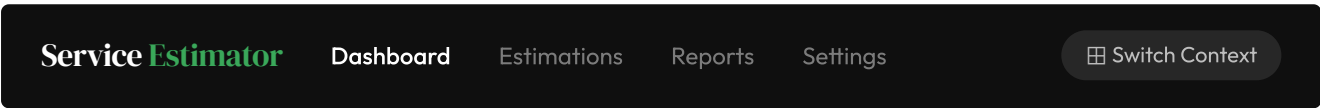
04 --

Context Switching & Navigation

Unified multi-app experience without cognitive overhead

Users work across 4 estimation domains but need a unified experience that doesn't feel like switching between separate applications. The solution: a single grid icon triggers an instant context swap — URL, nav, and branding update simultaneously while the shell stays constant.

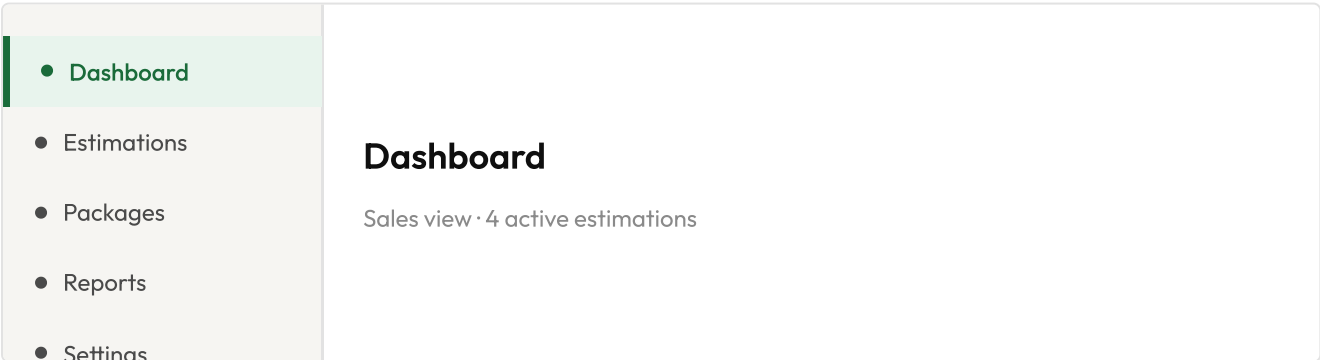
NAVIGATION SHELL



CONTEXT ROUTING

CONTEXT	ROUTE PREFIX	NAV COMPONENT	UNIQUE UI
Service Estimator	/estimator	NavMenu (Sales/Admin)	Kanban, Charts
Product Estimator	/products	ProductNavMenu	Addon slider
Service Management	/servicemanagement	SMNavMenu	Package clone
SMS Gateway	/smsestimator	SMSNavMenu	Tier tables

NAVIGATION ARCHITECTURE



DESIGN PRINCIPLE

Same nav shell, same typography, same component styles across all four contexts. Only navigation items and content change — the surrounding UI remains perfectly familiar regardless of which domain the user is in.

Form Interactions

Consistent, validated data entry with clear feedback at every step

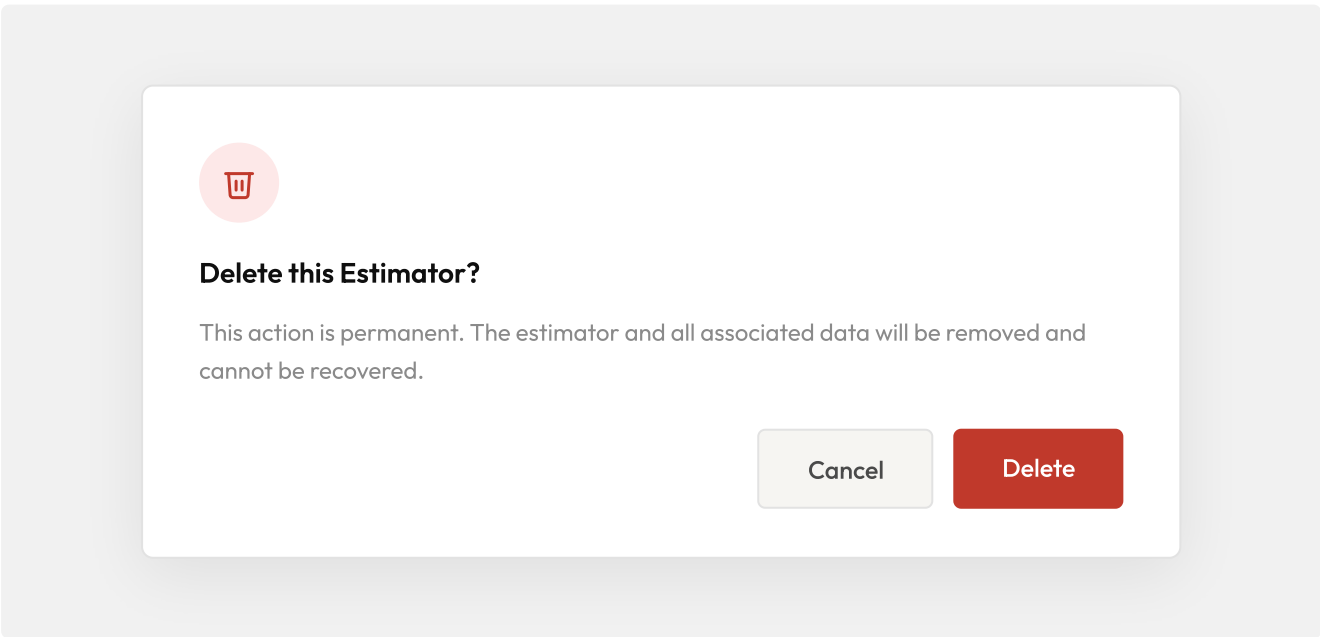
MODAL FORM PATTERN



UX CHARACTERISTICS

ASPECT	IMPLEMENTATION	USER BENEFIT
Validation	Real-time, field-level (Yup schema)	Errors shown as the user types — no surprise on submit
Error Display	Red helper text below invalid field	Contextual, not a separate error summary to hunt for
Loading State	Button disabled + inline spinner	Prevents double-submission and signals progress
Success	Green toast notification	Confirmation without blocking or interrupting workflow
API Error	Red toast with server message	User understands exactly what went wrong
Cancel	Always available, no confirmation	User never feels trapped in a form they don't want

DELETE CONFIRMATION PATTERN



Destructive actions always require explicit modal confirmation. The user must actively choose to proceed — preventing data loss from misclicks or momentary inattention.

Design System

Unified visual language powering all four application contexts

COLOR PALETTE

							
Primary #30af38	Primary Dark #278f2e	Primary Light #e8f4ed	Text Dark #444444	Text Mid #676767	Text Light #999999	Body BG #eaeaf0	Danger #dc3545

TYPOGRAPHY SCALE

ELEMENT	FONT	WEIGHT	SIZE
Page titles	Poppins	600 Semi-bold	h1–h3 scale
Section headings	Poppins	500 Medium	h4–h5 scale
Body text	Poppins	400 Regular	15px
Labels & captions	Poppins	400 Regular	13–15px

SHARED COMPONENTS

 ProtectedPage Permission-gated wrapper — shows content only if the user holds required access level	 SearchFilter Debounce search (300ms delay) — prevents excessive API calls during rapid typing
 Pagination Page size selector (12 / 24 / 36 / 48) plus page navigation — consistent across all list views	 DeleteModal Confirmation dialog with loading state — prevents accidental destructive actions across all contexts

33 DESIGN TOKENS	14 BOOTSTRAP OVERRIDES	17 SHARED COMPONENTS
---------------------	---------------------------	-------------------------

Context Comparison & Growth

How the four contexts compare and patterns enabling future expansion

CONTEXT COMPARISON MATRIX

Aspect	Service	Product	Svc Mgmt	SMS
ESTIMATION MODEL	Custom modules	Product catalog	Packages	Usage tiers
WORKFLOW	6-step wizard	Linear	List → Export	List → Export
UNIQUE UI	Kanban, Charts	Addon slider	Pkg clone	Tier tables
COMPLEXITY	HIGHEST	MEDIUM	MEDIUM	LOWEST
PAGES / ROUTES	35 / 13	55 / 10	40 / 9	39 / 6

EXPANSION PATTERNS

- + **New context:** Create pages, define routes, add nav menu — shared infrastructure works automatically
- + **New component:** Place in `src/components/` and all contexts can import immediately
- + **Brand change:** Update one SCSS token variable — entire application updates in one pass
- + **New form:** Follow existing Formik + Yup + Modal pattern for instant consistency

PRINCIPLES TO MAINTAIN

Preserve Context Switching

New features should live within their context's URL prefix and use the shared nav pattern. Never break the context boundary.

Extract, Don't Duplicate

If a component is used in 2+ places, extract it to shared components. One-off inline components are a maintenance liability.

Never Hardcode Values

Always use SCSS tokens for colors, spacing, and typography. One source of truth keeps the system consistent and changeable.

Handle Empty States

Every list must handle zero-data gracefully with the NoData component and a clear, actionable message for the user.